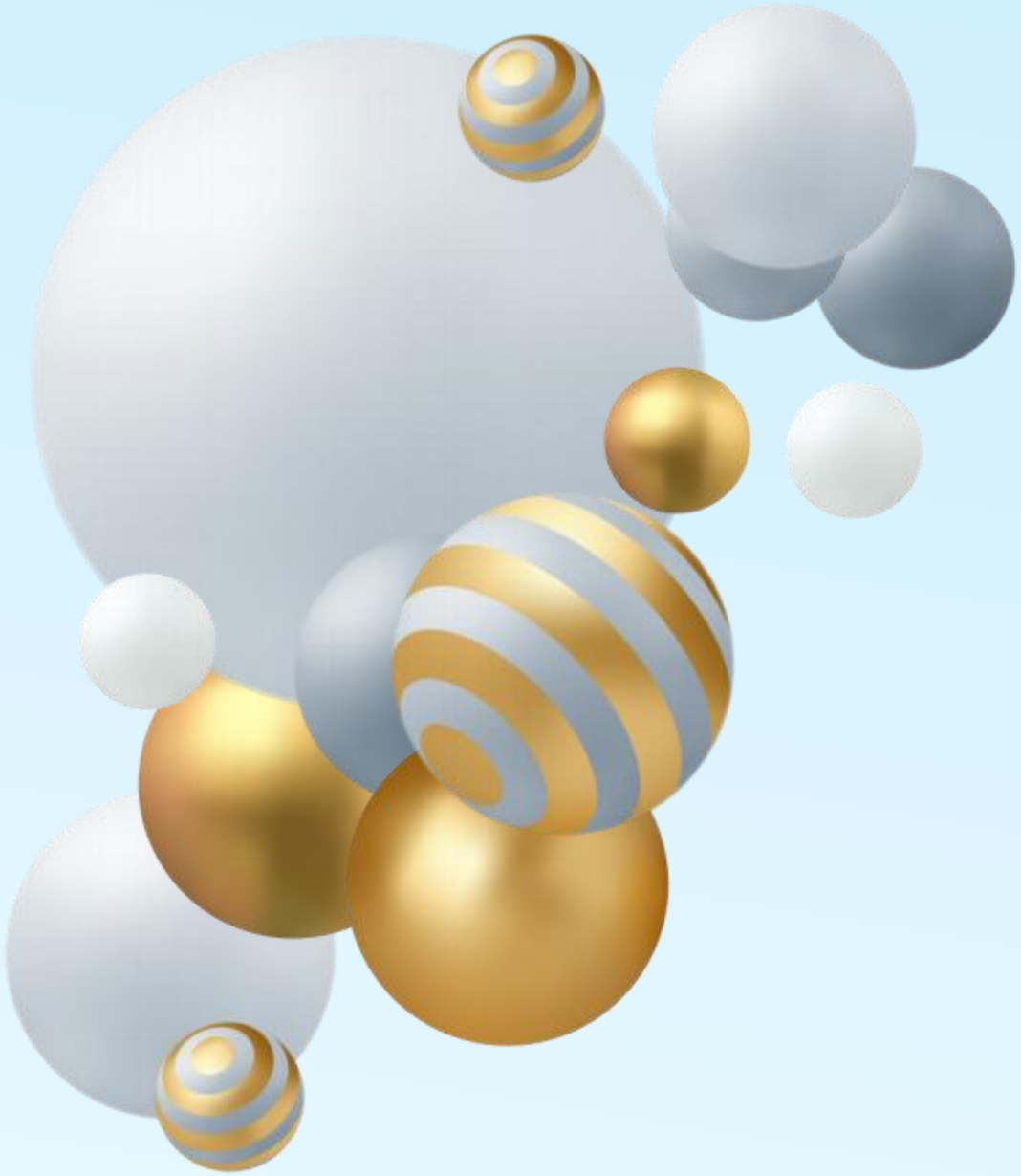
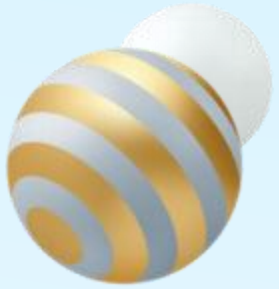


Computer Organization

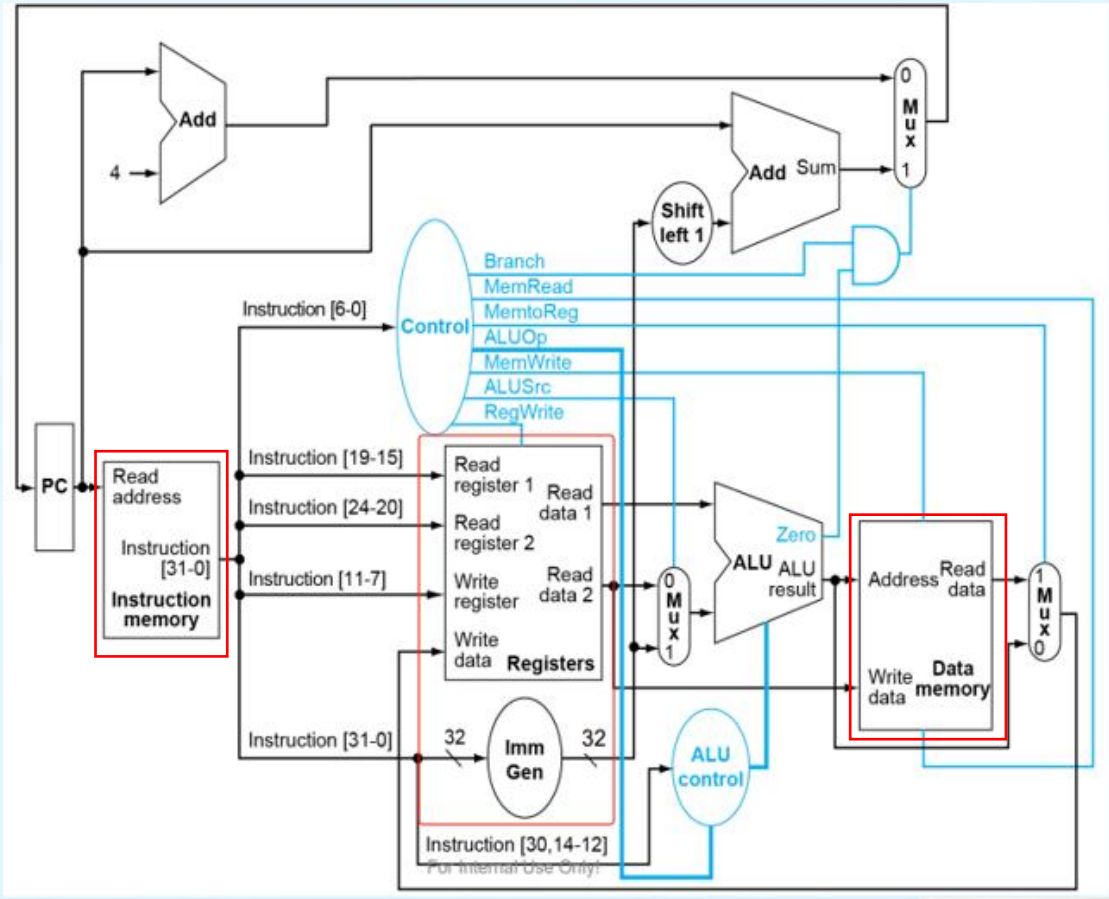
Lab10 CPU Design(1)

ISA, Control + Data
Block Memory,Decoder



Topic

- **CPU Design**(1)
  - ISA
  - CPU = Control Path + Data Path
  - Data Path(1) Memory & Registers
    - Memory implementation
    - Memory in CPU
      - Instruction Memory
      - Data Memory
    - Registers



RISC-V32I

RISC-V-Reference-Data.pdf

I type {imm[11:0], rs1, funct3, rd, opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
lb	**000_0011**	0		**03**/0/-
lh	**000_0011**	1		03/1/-
lw	**000_0011**	10		03/2/-
ld	**000_0011**	11		03/3/-
lbu	**000_0011**	100		03/4/-
lhu	**000_0011**	101		03/5/-
jalr	**110_0111**	0		**67/0/-**
addi	**001_0011**	0		**13/0/-**
slli	**001_0011**	1	0	13/1/0
slti	**001_0011**	10		13/2/-
sltiu	**001_0011**	11		13/3/-
xori	**001_0011**	100		13/4/-
srli	**001_0011**	101	0	13/5/0
srai	**001_0011**	101	010_0000	13/5/20
ori	**001_0011**	110		13/6/-
andi	**001_0011**	111		13/7/-

S type {imm[11:5], rs2, rs1, funct3, imm[4:0], opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
sb	**010_0011**	0		**23/**
sh	**010_0011**	1		
sw	**010_0011**	10		
sd	**010_0011**	11		

R type {func7, rs2, rs1, func3, rd, opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
add	**011_0011**	0	0	**33/**
sub	**011_0011**	0	010_0000	
sll	**011_0011**	1	0	
slt	**011_0011**	10		
sltu	**011_0011**	11		
xor	**011_0011**	100		
srl	**011_0011**	101	0	
sra	**011_0011**	101	010_0000	
or	**011_0011**	110		
and	**011_0011**	111		

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

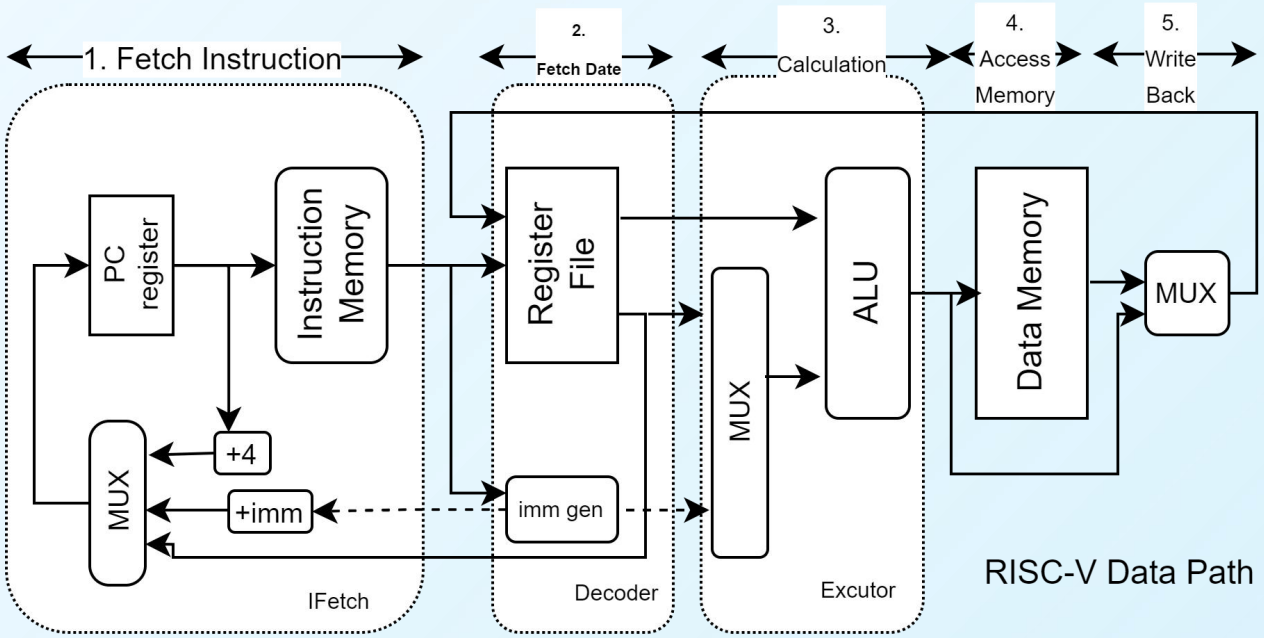
SB type {imm[12|10:5], rs2, rs1, funct3, imm[4:1|11], opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
beq	**110_0011**	0		**63/**
bne	**110_0011**	1		
blt	**110_0011**	100		
bge	**110_0011**	101		
bltu	**110_0011**	110		
bgeu	**110_0011**	111		

U type {imm[31:12], rd, opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
auipc	**001_0111**			**17/**
lui	**011_0111**			**37/**

UJ type {imm[20|10:1|11|19:12], rd, opcode}				
inst	**OPCODE**	**FUNCT3**	**FUNC7**	**hex**
jal	**110_1111**			**6f/**

Data Path(1)

Data Path:
The parts in CPU with componets which are involved to execute instructions.

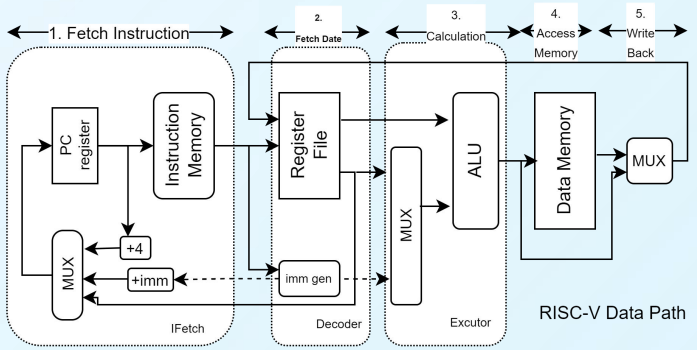


	1. Instruction fetch	2. Data Fetch	3. Instruction Execute	4. Memory Access	5. Register WriteBack
add[R]	Y	Y	Y		Y
addi[I]	Y	Y	Y		Y
sw[S]	Y	Y	Y	Y	
lw[I]	Y	Y	Y	Y	Y
branch[SB]	Y	Y	Y		
jalr[I]	Y	Y	Y		Y
jal [UJ]	Y	Y	Y		Y

Data Path(2)

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode



Module	How To Process	Instructions and Comments
IFetch	Determine how to update the value of PC register	1. pc+immediate(sign extended) (bne [**b**], jal[J]) 2. the value of the specified register (ra by default) (jalr [I]) 3. pc+4 (other instruction except branch, jal and jal)
Decoder	Determine whether to write register or not	lw [I], R type, U type vs beq([B]) vs sw([S])
	Get the source of data to be written Get the address of register to be written	1. Data memory (lw[I]) -> rd 2. ALU ([R]) -> rd 3. address of instruction (jal[J]) -> the specified register
	Determine whether to get immediate data from the instruction and expand it to 32bit	add([R]) vs addi([I]) vs lui([U]) vs sw([S]) vs beq([B]) vs jal([J])
DMemory	Determine whether to write memory or not	(sw[S]) vs lw[I]
	Get the source of data to be written	rs1 vs rs2 of registers (sw[S])
	Get the address of memory unit to be written	the output of ALU (sw[S])
ALU	Determine how to calculate the datas	add, sub, or, sll, sltiu, sra, slt, branch ...
	determin the source of one operand from register or immediate extended	R(register), I(sign extended immediate)

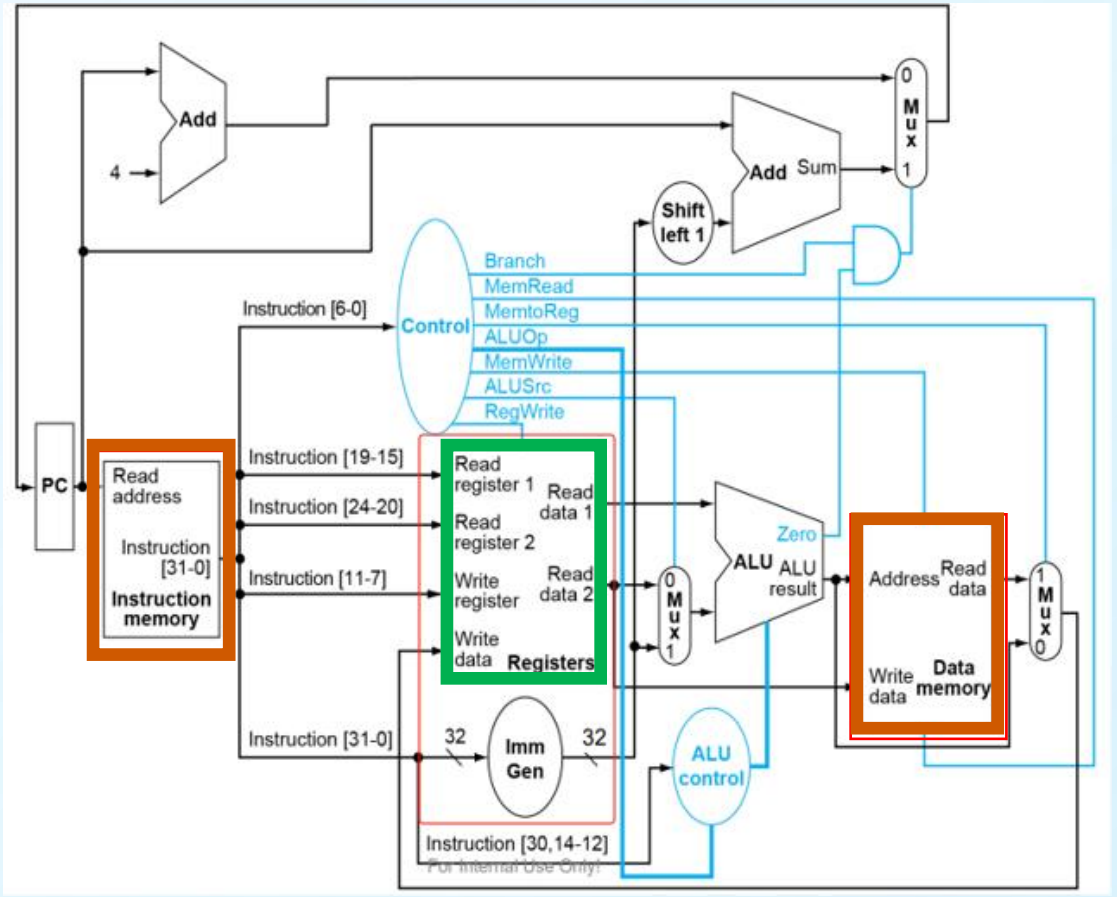
Memory Implementation in FPGA

Type	Physical resource	Features	Verilog inference
Block Memory	Using **the dedicated embedded memory block** within the FPGA	With a **large** capacity, it possesses deterministic timing characteristics and supports true dual-port read and write operations. suitable for medium-capacity data storage.	By declaring a large two-dimensional array and using synchronous read-write logic, the synthesizer will usually automatically infer it as a BRAM. use following method to mandatory specification: **(* ram_style = "block" *)** reg [x:0] mem [0:y];
Distributed Memory	Using **the lookup table (LUT) and flip-flops** in the logic unit of FPGA.	With a relatively **small** capacity, it is distributed throughout the entire logical array, resulting in extremely low access latency. suitable for small capacity and high-speed access, such as small FIFOs, coefficient tables, or state machine lookup tables.	When the defined array depth is shallow, the Vivado synthesizer tends to synthesize it as LUTRAM. use following method to mandatory specification: (* ram_style = "distributed" *) reg [x:0] mem [0:y];

Memory Implement in CPU

- Instruction Memory
- Data Memory
  - **large** capacity
  - **Block Memory**
  - **use verilog / vivado IP cores**

- Registers
  - **small** capacity
  - **Distributed Memory**
  - **use verilog**



Reminder:

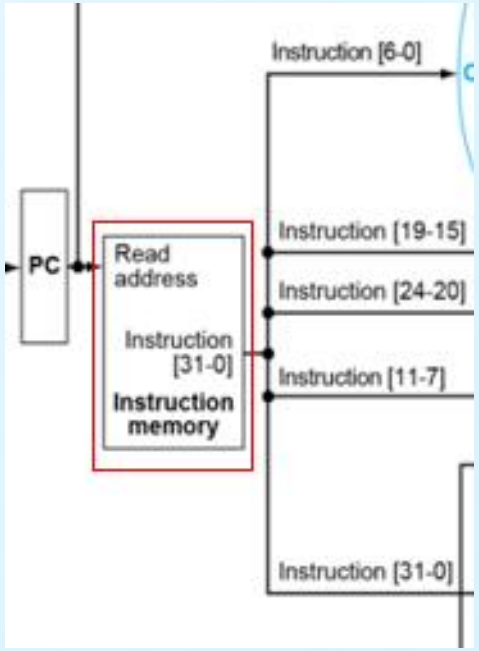
If using difftest (introduced in lab9) for testing, the overall CPU implementation does not use any IP cores.

Instruction Fetch(1) - Instruction Memory (1)

- **Circuit analysis**: Due to the involvement of storage, this circuit is a sequential logic circuit.
- **Circuit implement**: Block Memory (Vivado IPcores / Verilog)

- Questions:
  - ✓ **Q1: How to load instruction?**
  - ✓ **Q2: When and how to fetch instruction in a single cycle RISC-V CPU?**

- **Interface**
  - **inputs**
    - ✓ **clk**
    - ✓ **address**
    - ✓ **writeEnable ?**
    - ✓ **writeData ?**
  - **outputs**
    - ✓ **instruction**



Instruction Memory-Block Memory in Verilog(1)

```
module imem #(
    parameter DATA_WIDTH = 32, // Memory bit width
    parameter ADDR_WIDTH = 14, // Memory depth = 2^ADDR_WIDTH
    parameter INIT_FILE  = "prgrom32.txt"
)(
    input clka,
    input [ADDR_WIDTH-1:0] addra,
    output reg [DATA_WIDTH-1:0] douta
);

    (* ram_style = "block" *) reg [DATA_WIDTH-1:0] mem [0:(1<<ADDR_WIDTH)-1];

    integer i;
    initial begin
        for (i = 0; i < (1<<ADDR_WIDTH); i = i + 1)
            mem[i] = {DATA_WIDTH{1'b0}};
        if (INIT_FILE != "")
            $readmemh(INIT_FILE, mem);
    end

    // BRAM operations are synchronous; data is captured and output strictly at
    // the clock edge. (Same as the IP core functionality)
    always @(posedge clka) begin
            douta      <= mem[addra];
    end

endmodule
```

When conducting simulation in a Vivado project, if the 'INIT_FILE' does not contain path information, the initialization file should be placed in the default current directory of the simulator.

This default current directory of the simulator is:
project directory\project name.sim\sim_1\behav\xsim.

For example, if the project is located in the root directory of drive D and the project name is "p1", then the default current path of the simulator will be:
D:\p1\p1.sim\sim_1\behav\xsim

Instruction Memory-Block Memory in Verilog(2)

```
module tb_inst_mem2( );
reg clk;
reg [13:0] addr;
wire [31:0] dout;

imem urom(.clka(clk),.addra(addr),.douta(dout));

initial begin
    clk = 1'b0;
    forever  #5 clk = ~clk;
end

initial begin
    addr=14'h0;
    repeat(20) #17 addr = addr + 1;
    #20 $finish;
end

endmodule
```

Step1.Build the testbecn to verify the function of the Block Memory(ROM) in verilog.

Step2. copy the initial file “prgrom32.txt” to the ‘default current directory’ (Refer to the hints on page 16.)

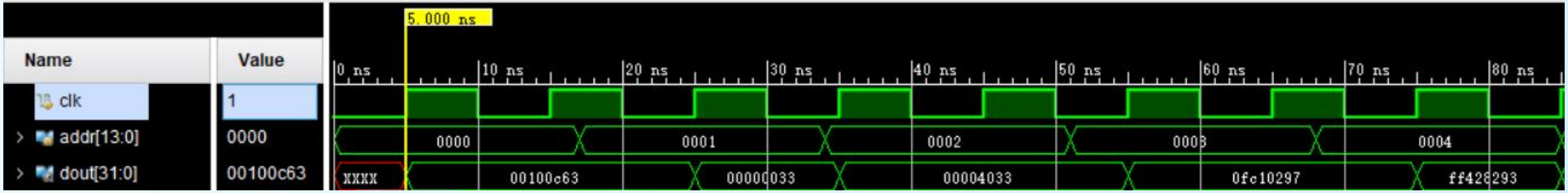
prgrom32.txt

C: > Users > sustech > project_1 > project_1.sim > sim_1 > behav > xsim > prgrom32.txt

```
1  00100c63
2  00000033
3  00004033
4  0fc10297
5  ff428293
6  00028083
7  00a00893
8  00000073
```

Step3. do the simulation based on the testbench.

Step4. Check the waveform generated by the simulation and the hex file(prgrom32.txt) which used to initialize the Block Memory(ROM) in verillog.



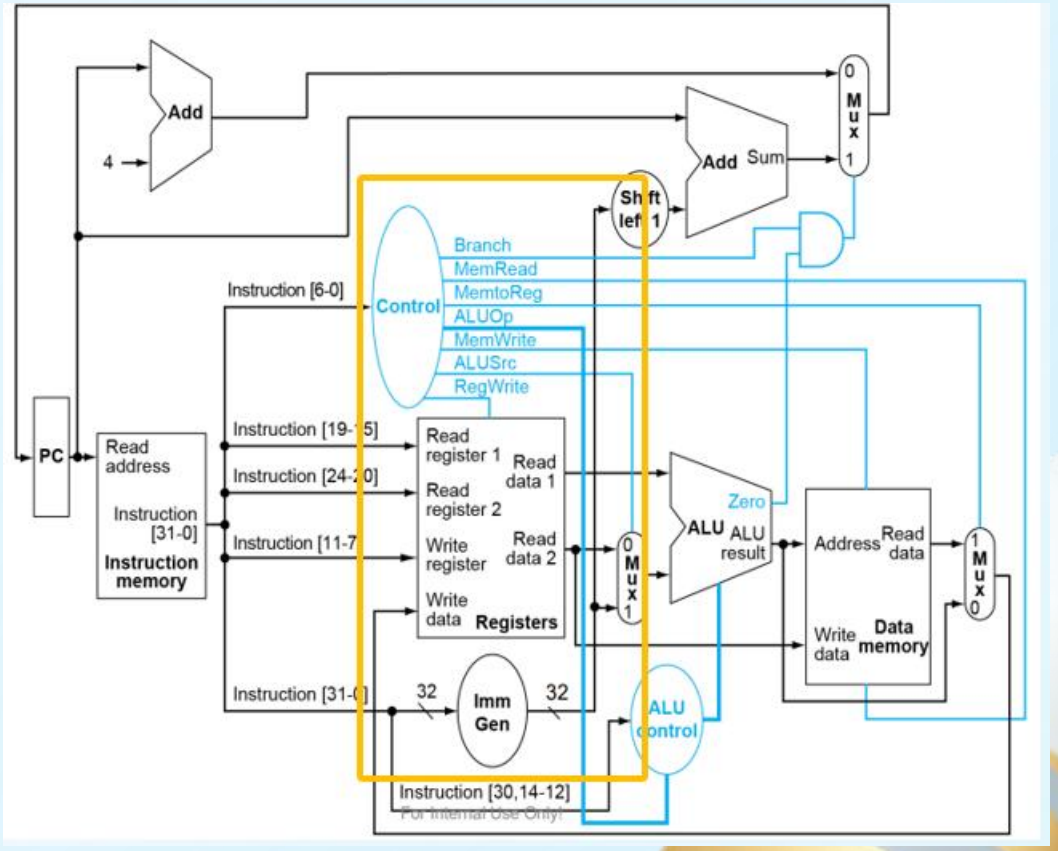
Instruction Analysis

- **Task 1: get information about the data from the instruction (Decoder in Data Path)**
  - address of **registers**: **rs2**(Instruction[24:20]), **rs1**(Instruction[19:15]) and **rd**(Instruction[11:7])
  - **shift mount**(instruction[24:20]) (R type and I type)
  - **immediate(12bits for I/S/SB, 20bits for U/UJ)**

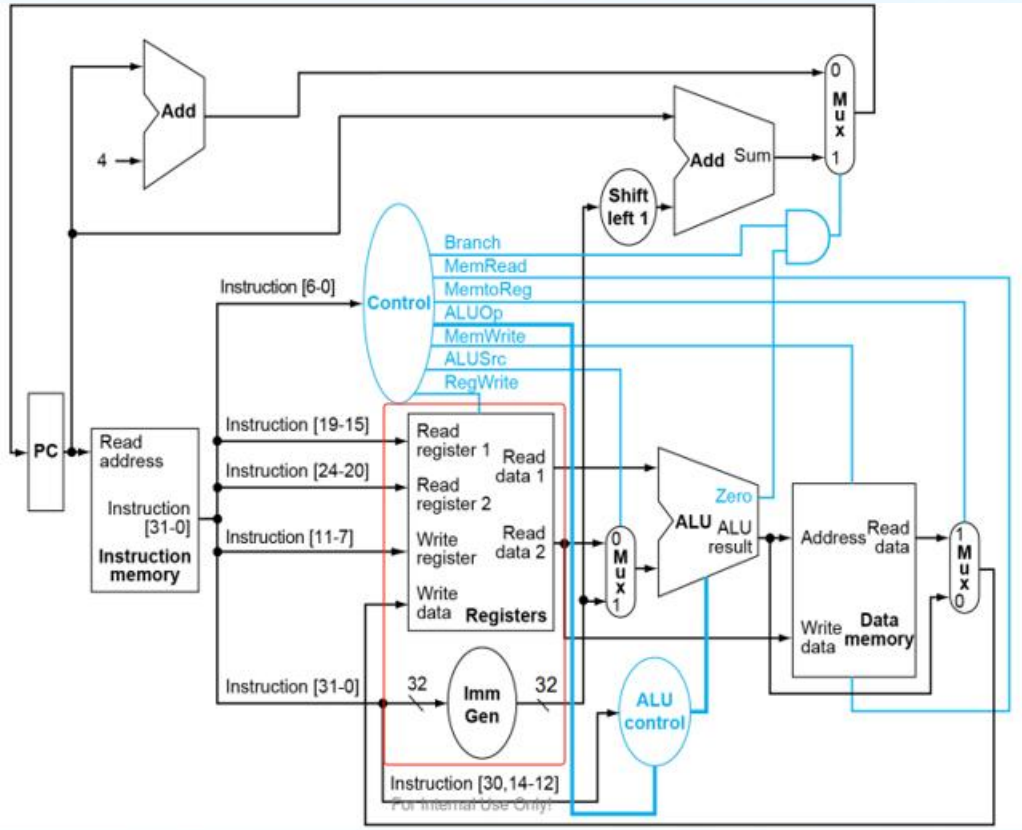
- **Task 2: generated control signals according to the instruction (Controller in Control Path)**
  - **get Operation code**(inst[**6:0**]) **and function code**(funct7,funct3) **in the instruction**
  - **generate control signals** to submodules of CPU

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode



Instruction Analysis(1)-Registers(1)



- **Get data** from the instruction directly or indirectly
  - opcode, function code : how to get data, where to get data
  - **immediate data** needs to be **signextended to 32bits for calculation.**
  - **data in the register**, the address of the register is coded in the instruction. e.g. rs2 = Instruction[24:20];
  - **data in the memory**, the **address** of the memory unit need to be calculated by ALU with **base address** stored in the **register** and **offset** as **immediate data in the instruction**

- **Read/Write data from/to Register File**

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode

Tips:
The submodule 'Imm Gen' has already been done in practice. It is suggested to instance 'Imm Gen' in the Decoder to speed up the coding.

Registers(2)

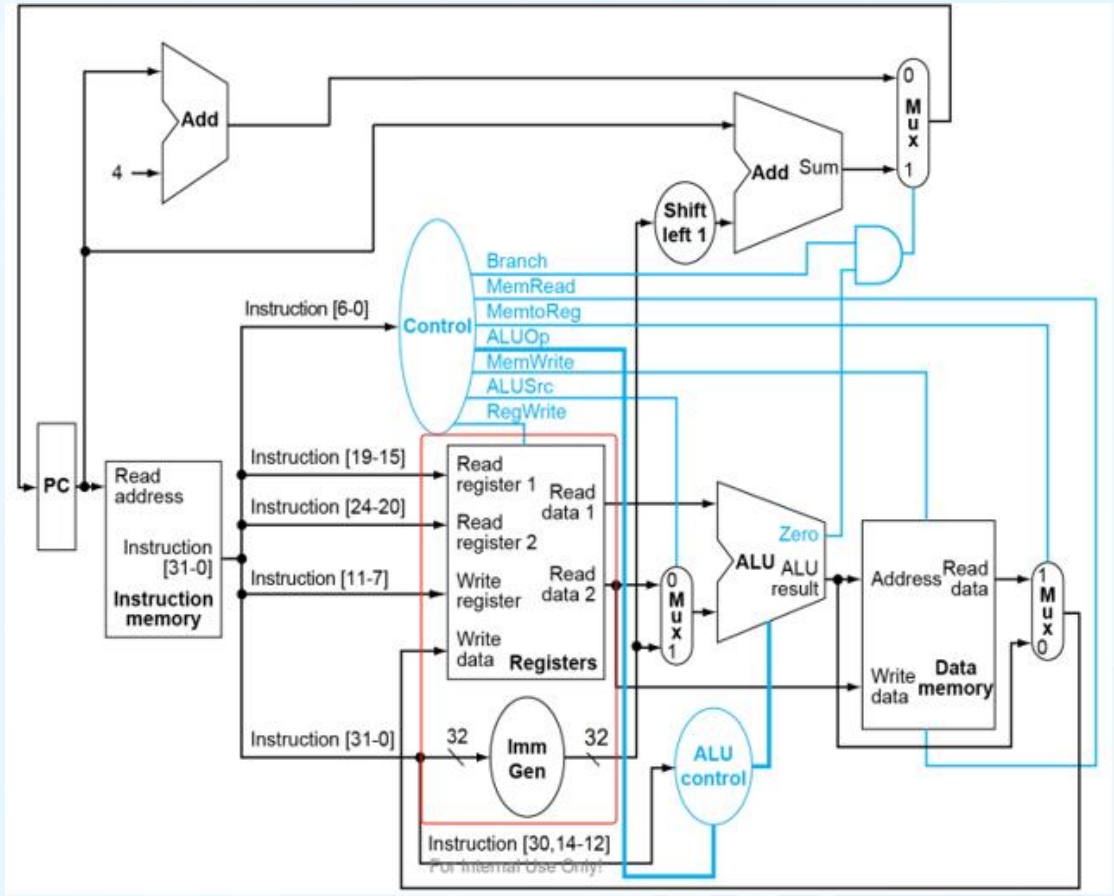
- **Registers/Register File**

  -Inputs

  - **read address**
    - [R] add: rs1,rs2
    - [B] beq : rs1, rs2
    - [I_calculate] addi: rs1
    - …
  - **write address**
    - [R] add: rd
    - [J] jal : rd/[31]
    - [I_calculate] addi: rd
    - …
  - **write data**
    - [R]/[I_calculate] add: data from alu_result
    - [I_load] lw: data from memory
    - …
  - **control signal**
    - [R]/[I]/[J] RegWrite
    - [J] jal
    - …

CORE INSTRUCTION FORMATS

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode



Registers(Register File)

-Outputs

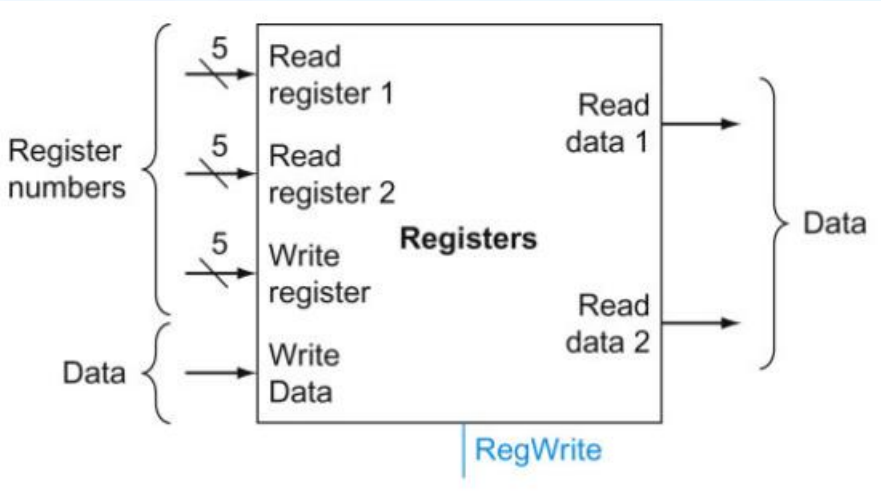
- read data1
- read data2
- extended Immi

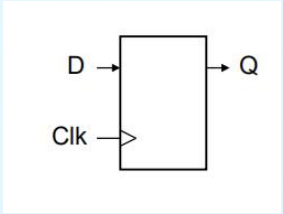
Registers(3)

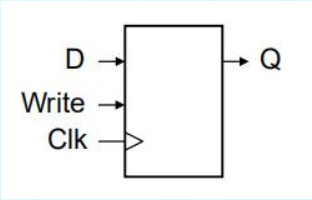
Registers/Register File:
Almost all the instructions need to read or write register file in CPU;
32 common registers with same bitwidth: 32

CORE INSTRUCTION FORMATS

	31–27	26–25	24–20	19–15	14–12	11–7	6–0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode
S	imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode
SB	imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]					rd	opcode
UJ	imm[20|10:1|11|19:12]					rd	opcode







Q1:
How to avoid the confliction between register read and register write?

Q2：Which kind of circuit is this register file, combinatorial circuit or sequential circuit?

Q3: How to determine the size of address bus on register file?

```
//verilog tips:
reg[31:0] register[0:31];
assign Rdata1 = register[Read register 1];
register[Write register] <= WriteData;
```

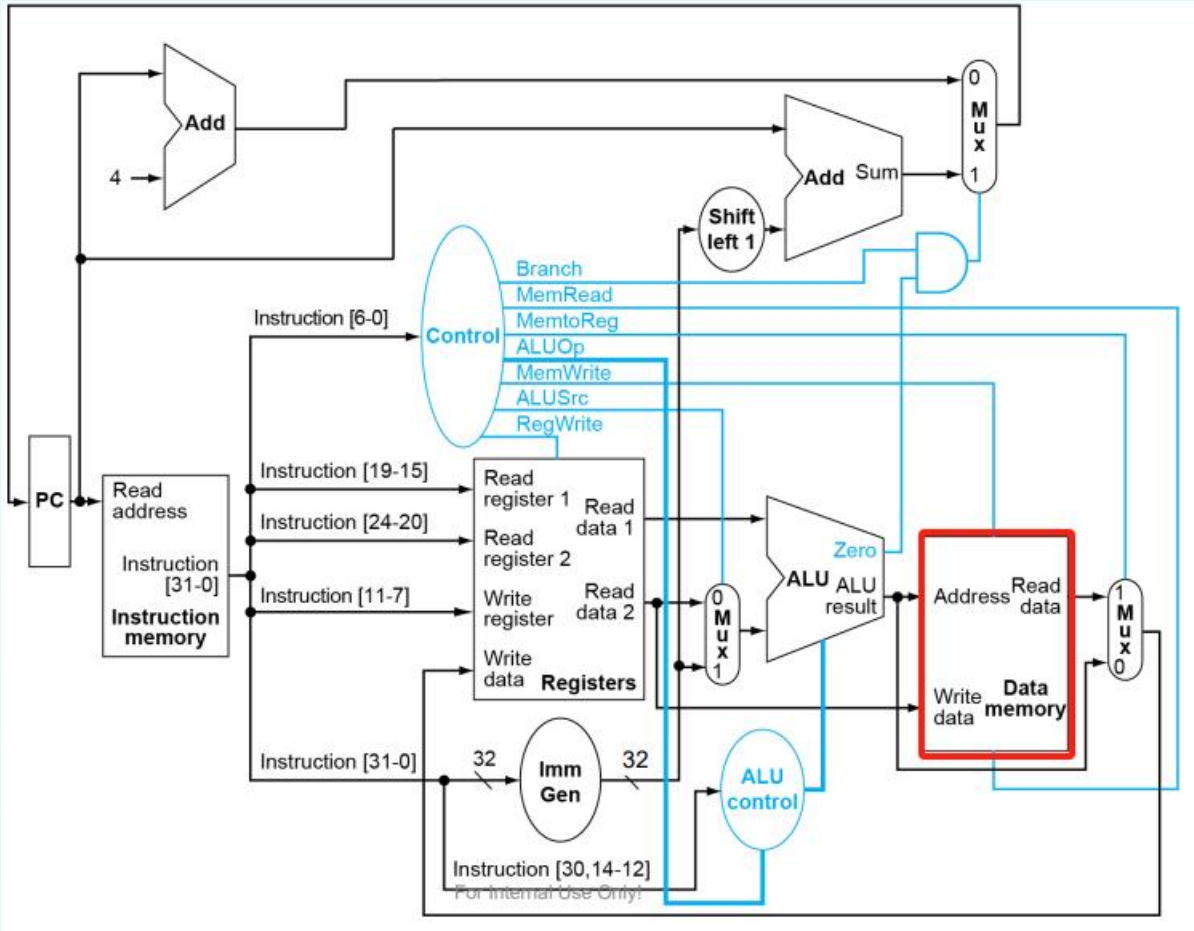
Instruction execution - Data Memory

- **Circuit analysis**: Due to the involvement of storage, this circuit is a sequential logic circuit.
  - ✓ **Q1: What's the bitwidth of address and data interface?**
  - ✓ **Q2: Would memory units be read and written simultaneously in a single cycle RISC-V CPU?**

- **inputs**
  - ✓ **clk**
  - ✓ **MemRead, MemWrite**
  - ✓ **Address**
  - ✓ **WriteData**

- **outputs**
  - ✓ **ReadData**

- **Implements:**
  - ✓ **Using Verilog** to implement the Data-memory.



Practice

1. **Design the registers read-write circuit in verilog, submit it to the educoder.net for testing.**

Task: implementation of a single-cycle RISC-V32I CPU internal register group read-write circuit using Verilog HDL. The circuit contains 32 32-bit wide registers (numbered 0~31), with register 0 always having a value of 0. It supports writing data to the target register when the write enable signal is valid at the clock rising edge. Data from two registers can be read at any time for output.

- ✓ Design a Verilog module named `regs`, which contains 32 general-purpose registers (register indices 0~31, with a bit width of 32 bits for each register). Except for register 0, which is always 0, all other registers are readable and writable.
- ✓ The input port `clk` of the module serves as the clock signal, with a bit width of 1bit; the input port `writeReg` serves as the write control signal, which is active at high level, with a bit width of 1bit.
- ✓ The input ports `rs1` and `rs2` of the module are subscripts of the source register, and the data read from them are sent to the output ports `readData1` and `readData2` respectively. `rd` is the subscript of the destination register. When the clock rises and `writeReg` is 1, the data in the input port `writeDate` is written into the destination register.

2. **Design the data memroy circuit in verilog, submit it to the educoder.net for testing.**

Task: implementation of a readable and writable block memory using Verilog HDL.

- ✓ The input port `clka` of the module serves as a clock signal with a bit width of 1bit; the input port `wea` serves as a write enable signal with a bit width of 1bit, which is active high; the input port `addra` serves as an address signal, with its bit width determined by the parameter `ADDR_WIDTH`; the input port `dina` serves as a write data signal, with its bit width determined by the parameter `DATA_WIDTH`; the output port `douta` serves as a read data signal, with its bit width determined by the parameter `DATA_WIDTH`.

- ✓ Inside the module, there is a block memory with a bit width of `DATA_WIDTH` and a depth of 2^ADDR_WIDTH. This memory can be initialized through `INIT_FILE`. If the file is empty or cannot be found, the values of all storage units are initialized to 0. At the rising edge of `clka`, if `wea` is 0, the data in the storage unit corresponding to the address `addra` is read and output from `douta`. At the rising edge of `clka`, if `wea` is 1, data is written to the storage unit corresponding to the address `addra`, and then the refreshed value is read from that storage unit (write first, then read).